Stash: setting work aside

Objectives

- Explain what git stash saves and what it leaves behind by default
- Use git stash, git stash list and git stash pop to switch tasks and come back

Teaching

Start with a guess

Sometimes you must drop what you are doing: an urgent bug needs you on another branch. Committing half-done work just to switch is messy. Git can set it aside instead.

Model

Practice repository Simulation: no real Git runs Working copy on feature app.js modified draft.md untracked Staging area next commit nothing staged Branches latest commit main C3 Fix login HEAD → feature C4 Add search Stash newest first empty

Recap

Recapgit stash sets aside changes to tracked files and returns them to the last commit, so you can switch.Untracked files stay unless you use git stash -u.git stash list shows entries; stash@{0} is the newest.git stash pop reapplies the newest entry to the current branch and removes it once it applies cleanly (after a conflict the entry stays until you drop it), so switch back first.Cheat sheetgit stashset aside tracked changesgit stash -ualso set aside untracked filesgit stash listshow saved entriesgit stash popreapply newest entry here and remove it (kept if it conflicts)Sources and review dateTeaching claims were checked on 2026-10-06 (UTC) against live official documentation: the Git reference manual on git-scm.com (latest source release 2.56.0, released 2026-09-28). Simulated command output is illustrative: its exact wording was not checked and varies with Git version and settings.git-stash: descriptiongit-stash: options (--include-untracked)git-switch: descriptionNext topic: Remotes: SSH keys, clone, push, fetch vs pull

Cheat sheet

Cheat sheetgit stashset aside tracked changesgit stash -ualso set aside untracked filesgit stash listshow saved entriesgit stash popreapply newest entry here and remove it (kept if it conflicts)

Sources and review dateTeaching claims were checked on 2026-10-06 (UTC) against live official documentation: the Git reference manual on git-scm.com (latest source release 2.56.0, released 2026-09-28). Simulated command output is illustrative: its exact wording was not checked and varies with Git version and settings.git-stash: descriptiongit-stash: options (--include-untracked)git-switch: description

Sources

Sources and review dateTeaching claims were checked on 2026-10-06 (UTC) against live official documentation: the Git reference manual on git-scm.com (latest source release 2.56.0, released 2026-09-28). Simulated command output is illustrative: its exact wording was not checked and varies with Git version and settings.git-stash: descriptiongit-stash: options (--include-untracked)git-switch: description

Static interaction alternatives

predict

Prediction (not scored). You have an unfinished edit to app.js on feature. You run git stash. What happens to the edit? Options: Saved on a stack; the file goes back to the last commit; Committed to feature; Deleted; Moved to main. Answer: Saved on a stack; the file goes back to the last commit. Saved on a stack; the file goes back to the last commit: git stash saves changes to tracked files on a stack called the stash and cleans them out of your working copy. Committed to feature: Attractive, but stash makes no commit on your branch; the branch history does not change. Deleted: Nothing is lost: the change is kept in the stash until you bring it back. Moved to main: Stash is not tied to switching; it just sets the change aside.

terminal

Practice terminal (simulated; nothing runs on your computer). Task: Your app.js edit on feature is unfinished, and an urgent fix waits on main. Visit main, then return and resume. Goals: Set aside the app.js edit; Visit main; Back on feature, edit restored, stash empty. One sequence that meets every goal: git stash, then git switch main, then git switch feature, then git stash pop.

mcq

Check (one retry, then the answer is shown). You have a modified tracked file cart.js and a new untracked file todo.txt. You run git stash. What does git status show next? Options: Only todo.txt, as untracked; Nothing: working tree clean; cart.js modified; Both cart.js and todo.txt. Answer: Only todo.txt, as untracked. Only todo.txt, as untracked: Right: the cart.js change went into the stash; untracked todo.txt stays because plain git stash skips untracked files. Nothing: working tree clean: That needs git stash -u, which also takes untracked files. cart.js modified: The tracked change to cart.js is exactly what stash takes away. Both cart.js and todo.txt: Stash did remove the cart.js change; only todo.txt remains. Retry question: The model shows style.css modified (tracked) and notes.md untracked. After plain git stash, where is each file's change? Options: Stash: style.css. Working copy: notes.md; Stash: style.css and notes.md; Stash: notes.md. Working copy: style.css; Neither; both are committed. Answer: Stash: style.css. Working copy: notes.md. Stash: style.css. Working copy: notes.md: Right: tracked change stashed, untracked file left behind. Stash: style.css and notes.md: Only git stash -u would take the untracked notes.md. Stash: notes.md. Working copy: style.css: Backwards: stash takes tracked changes, not untracked ones. Neither; both are committed: Stash never commits to your branch.